1. Create a folder in the GP/rtl directory next to the one that contained the axi peripheral integrated butterfly.
2. Paste the files in DMA_BUTTERFLY_TCDM_SLAVE on Github repo in the created folder.
3. Use reggen tool to generate registers package and reg top

Export Reggen path in terminal:

export REGGEN=$HOME/GP/.bender/git/checkouts/register_interface-1e5f22ae6b2b5b5d/vendor/lowrisc_opentitan/util/regtool.py

then generate register files:

python3 $REGGEN -r rtl/DMA_BUTTERFLY_TCDM_SLAVE/butterfly.hjson --outdir rtl/DMA_BUTTERFLY_TCDM_SLAVE/

To generate header:

python3 $REGGEN -D rtl/DMA_BUTTERFLY_TCDM_SLAVE/butterfly.hjson > pulp-runtime/include/DMA_butterfly_regs.h

4. Now you can paste the following files (yareet 3ashan mayfr23sh) in the paths or do the integration manually to avoid naming conflicts

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Pulp_soc

Modifications:

Under signals declarations

```
355
356     // CHANGE HERE TCDM DMA
357   // ================================================================================
358   // Butterfly TCDM Interface Bus
359   // ================================================================================
360   XBAR_TCDM_BUS s_bfly_tcdm_bus();
361
362   // END
```

Copy code lines from the file pulp_soc.sv from github

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Pulp_soc

Modifications:

Instantiate wrapper(top) under the soc peripherals:

Copy code lines from the file pulp_soc.sv from github

```
619
620     //*********************************************************
621     //******************** SOC PERIPHERALS ********************
622     //*********************************************************
623
624     //CHANGE HERE DMA TCDM ADD INSTANTIATION
625 // ============================================================================
626   // Butterfly Accelerator (TCDM Slave)
627   // ============================================================================
628   butterfly_tcdm_top #(
629       .ADDR_WIDTH(32),
630       .DATA_WIDTH(32)
631   ) i_butterfly_tcdm (
632       .clk_i          (s_soc_clk),
633       .rst_ni         (s_soc_rstn),
634       .test_mode_i    (dft_test_mode_i),
635
636       // Pass the interface directly! No more unpacking errors.
637       .tcdm_slave     (s_bfly_tcdm_bus)
638   );
639   // END
```

Copy code lines from the file pulp_soc.sv from github

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Pulp_soc

Modifications:

```
899     soc_interconnect_wrap #(
900       .NR_HWPE_PORTS(NB_HWPE_PORTS),
901       .NR_L2_PORTS(NB_L2_BANKS),
902       .AXI_IN_ID_WIDTH(AXI_ID_IN_WIDTH),
903       .AXI_USER_WIDTH(AXI_USER_WIDTH)
904     ) i_soc_interconnect_wrap (
905         .clk_i                ( s_soc_clk           ),
906         .rst_ni               ( s_soc_rstn          ),
907         .test_en_i            ( dft_test_mode_i     ),
908         .tcdm_fc_data         ( s_lint_fc_data_bus  ),
909         .tcdm_fc_instr        ( s_lint_fc_instr_bus ),
910         .tcdm_udma_rx         ( s_lint_udma_rx_bus  ),
911         .tcdm_udma_tx         ( s_lint_udma_tx_bus  ),
912         .tcdm_debug           ( s_lint_debug_bus    ),
913         .tcdm_hwpe            ( s_lint_hwpe_bus     ),
914         .axi_master_plug      ( s_data_in_bus       ),
915         .axi_slave_plug       ( s_data_out_bus      ),
916         .apb_peripheral_bus   ( s_apb_periph_bus    ),
917         .l2_interleaved_slaves ( s_mem_l2_bus       ),
918         .l2_private_slaves    ( s_mem_l2_pri_bus    ),
919         .boot_rom_slave       ( s_mem_rom_bus       ),
920         .wide_alu_slave         ( s_wide_alu_bus    ),
921         // CHANGE AXI BUTTERFLY
922         .butterfly_slave        ( s_butterfly_bus ),
923         // END
924
925         //CHANGE TCDM DMA
926 // TCDM DMA Connection
927         .butterfly_tcdm_slave  ( s_bfly_tcdm_bus      )  // Your NEW TCDM integration
928       // END
929         );
```

Copy code lines from the file pulp_soc.sv from github

---

5. Now, Soc interconnect wrap you can paste the following files (yareet 3ashan mayfr23sh) in the paths or do the integration manually to avoid naming conflicts

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Soc_interconnect_wrap

Modifications:

In the module port list :

```
XBAR_TCDM_BUS.Master     l2_private_slaves[2], // Connects to core-private memory banks
XBAR_TCDM_BUS.Master     boot_rom_slave, //Connects to the bootrom

// CHANGE HERE TCDM DMA
// Add the new TCDM port for the Butterfly using the standard interface!
XBAR_TCDM_BUS.Master     butterfly_tcdm_slave,
// END

//  add a new AXI port for the Wide ALU.
AXI_BUS.Master wide_alu_slave,
//  add a new AXI port for the Butterfly.It is a .Master here because the crossbar acts as the master sending data to your IP's slave port
AXI_BUS.Master butterfly_slave  );
```

Copy code lines from the file Soc_interconnect_wrap from github

---

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Soc_interconnect_wrap

Modifications:

Addressing rules (Note I am leaving the original code commented just for easier recovery in case it explodes, delete if you want)

```
100
101     ////////////////////////////////////////
102     // Address Rules for the interconnect //
103     ////////////////////////////////////////
104     localparam NR_RULES_L2_DEMUX = 4; // Increased from 3 to add DMA Butterfly routing
105     //Everything that is not routed to port 1 or 2 ends up in port 0 by default
106     /*localparam addr_map_rule_t [NR_RULES_L2_DEMUX-1:0] L2_DEMUX_RULES = '{
107        '{ idx: 1 , start_addr: `SOC_MEM_MAP_PRIVATE_BANK0_START_ADDR , end_addr: `SOC_MEM_MAP_PRIVATE_BANK1_END_ADDR} , //Both , bank0 and bank1 are in the
    same address block
108        '{ idx: 1 , start_addr: `SOC_MEM_MAP_BOOT_ROM_START_ADDR      , end_addr: `SOC_MEM_MAP_BOOT_ROM_END_ADDR}      ,
109        '{ idx: 2 , start_addr: `SOC_MEM_MAP_TCDM_START_ADDR          , end_addr: `SOC_MEM_MAP_TCDM_END_ADDR }};*/
110        //Everything that is not routed to port 1 or 2 ends up in port 0 by default
111     localparam addr_map_rule_t [NR_RULES_L2_DEMUX-1:0] L2_DEMUX_RULES = '{
112        '{ idx: 1 , start_addr: `SOC_MEM_MAP_PRIVATE_BANK0_START_ADDR , end_addr: `SOC_MEM_MAP_PRIVATE_BANK1_END_ADDR} ,
113        '{ idx: 1 , start_addr: `SOC_MEM_MAP_BOOT_ROM_START_ADDR      , end_addr: `SOC_MEM_MAP_BOOT_ROM_END_ADDR}      ,
114        '{ idx: 2 , start_addr: `SOC_MEM_MAP_TCDM_START_ADDR          , end_addr: `SOC_MEM_MAP_TCDM_END_ADDR },
115
116        // ADD THIS LINE: Route our TCDM address space into the Contiguous Crossbar (idx: 1)
117        '{ idx: 1 , start_addr: `SOC_MEM_MAP_BFLY_TCDM_START_ADDR     , end_addr: `SOC_MEM_MAP_BFLY_TCDM_END_ADDR}
118     };
119
```

Copy code lines from the file Soc_interconnect_wrap from github

---

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Soc_interconnect_wrap

Modifications:

L2 demux Addressing rules (Note I am leaving the original code commented just for easier recovery in case it explodes, delete if you want)

```
123
124 localparam NR_RULES_CONTIG_CROSSBAR = 4; // Increased from 3 to add Butterfly TCDM
125 localparam addr_map_rule_t [NR_RULES_CONTIG_CROSSBAR-1:0] CONTIGUOUS_CROSSBAR_RULES = '{
126         '{ idx: 0 , start_addr: `SOC_MEM_MAP_PRIVATE_BANK0_START_ADDR , end_addr: `SOC_MEM_MAP_PRIVATE_BANK0_END_ADDR} ,
127         '{ idx: 1 , start_addr: `SOC_MEM_MAP_PRIVATE_BANK1_START_ADDR , end_addr: `SOC_MEM_MAP_PRIVATE_BANK1_END_ADDR} ,
128         '{ idx: 2 , start_addr: `SOC_MEM_MAP_BOOT_ROM_START_ADDR      , end_addr: `SOC_MEM_MAP_BOOT_ROM_END_ADDR},
129         // Add the Butterfly TCDM Rule here:
130         '{ idx: 3 , start_addr: `SOC_MEM_MAP_BFLY_TCDM_START_ADDR     , end_addr: `SOC_MEM_MAP_BFLY_TCDM_END_ADDR}
131     };
132
```

Copy code lines from the file Soc_interconnect_wrap from github

---

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Soc_interconnect_wrap

Modifications:

Under instantiate interconnect

```
185     //Synopsys 2019.3 has a bug; It doesn't handle expressions for array indices on the left-hand side of assignments.
186     // Using a macro instead of a package parameter is an ugly but necessary workaround.
187     // E.g. assign a[param+i] = b[i] doesn't work, but assign a[i] = b[i-param] does.
188     `define NR_SOC_TCDM_MASTER_PORTS 5
189     for (genvar i = 0; i < 4; i++) begin
190         `TCDM_ASSIGN_INTF(master_ports[`NR_SOC_TCDM_MASTER_PORTS + i], axi_bridge_2_interconnect[i])
191     end
192
193     XBAR_TCDM_BUS contiguous_slaves[4](); // Increased from 3
194     `TCDM_ASSIGN_INTF(l2_private_slaves[0], contiguous_slaves[0])
195     `TCDM_ASSIGN_INTF(l2_private_slaves[1], contiguous_slaves[1])
196     `TCDM_ASSIGN_INTF(boot_rom_slave, contiguous_slaves[2])
197     // Add the assignment for your new Butterfly port:
198     `TCDM_ASSIGN_INTF(butterfly_tcdm_slave, contiguous_slaves[3])
199
```

Copy code lines from the file soc_interconnect_wrap.sv from github

---

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Soc_interconnect_wrap

Modifications:

Under instantiate interconnect

```
185     //Synopsys 2019.3 has a bug; It doesn't handle expressions for array indices on the left-hand side of assignments.
186     // Using a macro instead of a package parameter is an ugly but necessary workaround.
187     // E.g. assign a[param+i] = b[i] doesn't work, but assign a[i] = b[i-param] does.
188     `define NR_SOC_TCDM_MASTER_PORTS 5
189     for (genvar i = 0; i < 4; i++) begin
190         `TCDM_ASSIGN_INTF(master_ports[`NR_SOC_TCDM_MASTER_PORTS + i], axi_bridge_2_interconnect[i])
191     end
192
193     XBAR_TCDM_BUS contiguous_slaves[4](); // Increased from 3
194     `TCDM_ASSIGN_INTF(l2_private_slaves[0], contiguous_slaves[0])
195     `TCDM_ASSIGN_INTF(l2_private_slaves[1], contiguous_slaves[1])
196     `TCDM_ASSIGN_INTF(boot_rom_slave, contiguous_slaves[2])
197     // Add the assignment for your new Butterfly port:
198     `TCDM_ASSIGN_INTF(butterfly_tcdm_slave, contiguous_slaves[3])
199
```

Copy code lines from the file soc_interconnect_wrap.sv from github

---

Path:

/GP/.bender/git/checkouts/pulp_soc-c519334bd3ac5582/rtl/pulp_soc

File:

Soc_interconnect_wrap

Modifications:

Under instantiate interconnect

```
2     //Interconnect instantiation
3     soc_interconnect #(
4                         .NR_MASTER_PORTS(pkg_soc_interconnect::NR_TCDM_MASTER_PORTS), // FC instructions, FC data, uDMA RX, uDMA TX, debug access, 4 four 64-
  bit
5                                                 // axi plug
6                         .NR_MASTER_PORTS_INTERLEAVED_ONLY(NR_HWPE_PORTS), // HWPEs (PULP accelerators) only have access
7                                                                           // to the interleaved memory region
8                         .NR_ADDR_RULES_L2_DEMUX(NR_RULES_L2_DEMUX),
9                         .NR_SLAVE_PORTS_INTERLEAVED(NR_L2_PORTS), // Number of interleaved memory banks
0                         .NR_ADDR_RULES_SLAVE_PORTS_INTLVD(NR_RULES_INTERLEAVED_REGION),
1
2                         // CHANGE TCDM DMA
3                         .NR_SLAVE_PORTS_CONTIG(4), // Bootrom + private memory banks + Butterfly
4                         // END
5
6                         .NR_ADDR_RULES_SLAVE_PORTS_CONTIG(NR_RULES_CONTIG_CROSSBAR),
7                         .NR_AXI_SLAVE_PORTS(4), // 1 for AXI to cluster, 1 for SoC peripherals (converted to APB) ,additional one for Wide_Alu
  Accelerator,additional 1 for Butterfly
8                         .NR_ADDR_RULES_AXI_SLAVE_PORTS(NR_RULES_AXI_CROSSBAR),
9                         .AXI_MASTER_ID_WIDTH(1), //Doesn't need to be changed. All axi masters in the current
0                                                  //interconnect come from a TCDM protocol converter and thus do not have and AXI ID.
1                                                  //However, the unerlaying IPs do not support an ID lenght of 0, thus we use 1.
2                         .AXI_USER_WIDTH(AXI_USER_WIDTH)
3                         ) i_soc_interconnect (
4                                               .clk_i,
5                                               .rst_ni,
6                                               .test_en_i,
7                                               .master_ports(master_ports),
8                                               .master_ports_interleaved_only(tcdm_hwpe),
9                                               .addr_space_l2_demux(L2_DEMUX_RULES),
0                                               .addr_space_interleaved(INTERLEAVED_ADDR_SPACE),
1                                               .interleaved_slaves(l2_interleaved_slaves),
2                                               .addr_space_contiguous(CONTIGUOUS_CROSSBAR_RULES),
3                                               .contiguous_slaves(contiguous_slaves).
```

Copy code lines from the file soc_interconnect_wrap.sv from github

---

6. Now we specify the memory region

Path:

/GP/rtl/includes

File:

Soc_mem_map.svh

Modifications:

```
21 // DMA TCDM BUTTERFLY
22 `define SOC_MEM_MAP_TCDM_START_ADDR          32'h1C01_0000
23 `define SOC_MEM_MAP_TCDM_END_ADDR            32'h1C09_0000
24 // END
25 `define SOC_MEM_MAP_TCDM_ALIAS_START_ADDR    32'h0000_0000
26 `define SOC_MEM_MAP_TCDM_ALIAS_END_ADDR      32'h0010_0000
27
28 `define SOC_MEM_MAP_PRIVATE_BANK0_START_ADDR 32'h1C00_0000
29 `define SOC_MEM_MAP_PRIVATE_BANK0_END_ADDR   32'h1C00_8000
30
31 `define SOC_MEM_MAP_PRIVATE_BANK1_START_ADDR 32'h1C00_8000
32 `define SOC_MEM_MAP_PRIVATE_BANK1_END_ADDR   32'h1C01_0000
33
34 `define SOC_MEM_MAP_BOOT_ROM_START_ADDR      32'h1A00_0000
35 `define SOC_MEM_MAP_BOOT_ROM_END_ADDR        32'h1A04_0000
36
37 `define SOC_MEM_MAP_AXI_PLUG_START_ADDR      32'h1000_0000
38 `define SOC_MEM_MAP_AXI_PLUG_END_ADDR        32'h1040_0000
39
40 `define SOC_MEM_MAP_PERIPHERALS_START_ADDR   32'h1A10_0000
41 `define SOC_MEM_MAP_PERIPHERALS_END_ADDR     32'h1A40_0000
42 // Wide ALU, Large enough to cover the reggen-generated register file
43 `define SOC_MEM_MAP_WIDE_ALU_START_ADDR 32'h1A40_0000
44 `define SOC_MEM_MAP_WIDE_ALU_END_ADDR   32'h1A40_1000
45 // Butterfly, Large enough to cover the reggen-generated register file
46 `define SOC_MEM_MAP_BUTTERFLY_START_ADDR 32'h1A40_1000
47 `define SOC_MEM_MAP_BUTTERFLY_END_ADDR   32'h1A40_2000
```

Copy code lines:

// DMA TCDM BUTTERFLY

`define SOC_MEM_MAP_TCDM_START_ADDR 32'h1C01_0000

`define SOC_MEM_MAP_TCDM_END_ADDR 32'h1C09_0000

// END

---

7. Now we update bender.yml order of files is important:

```
 98  # --- The Math Core ---
 99  - rtl/DMA_BUTTERFLY_TCDM_SLAVE/convround.sv
100  - rtl/DMA_BUTTERFLY_TCDM_SLAVE/simple_twiddle_rom_bank.sv
101  - rtl/DMA_BUTTERFLY_TCDM_SLAVE/simple_butterfly.sv
102
103  # --- The New TCDM Integration ---
104  - rtl/DMA_BUTTERFLY_TCDM_SLAVE/butterfly_reg_pkg.sv
105  - rtl/DMA_BUTTERFLY_TCDM_SLAVE/butterfly_reg_top.sv
106  - rtl/DMA_BUTTERFLY_TCDM_SLAVE/butterfly_tcdm_top.sv
107
108  # --- SIMPLE BUTTERFLY ---
109    # --- The Math Core ---
110 # - rtl/simple_butterfly/convround.sv
111  #- rtl/simple_butterfly/simple_twiddle_rom_bank.sv
112  #- rtl/simple_butterfly/simple_butterfly.sv
113
114  - rtl/simple_butterfly/simple_butterfly_reg_pkg.sv
115  - rtl/simple_butterfly/simple_butterfly_reg_top.sv
116  - rtl/simple_butterfly/simple_butterfly_top.sv
117  # Butterfly
118  #- rtl/butterfly/butterfly_reg_pkg.sv
119  #- rtl/butterfly/convround.sv
120  #- rtl/butterfly/bimpy.sv
121  #- rtl/butterfly/longbimpy.sv
122  #- rtl/butterfly/butterfly.sv
123  #- rtl/butterfly/butterfly_reg_top.sv
124  #- rtl/butterfly/butterfly_top.sv
125
```

Copy code lines, if you won't use the axi again you can comment it:

```
# --- The Math Core ---
- rtl/DMA_BUTTERFLY_TCDM_SLAVE/convround.sv
- rtl/DMA_BUTTERFLY_TCDM_SLAVE/simple_twiddle_rom_bank.sv
- rtl/DMA_BUTTERFLY_TCDM_SLAVE/simple_butterfly.sv


# --- The New TCDM Integration ---
- rtl/DMA_BUTTERFLY_TCDM_SLAVE/butterfly_reg_pkg.sv
- rtl/DMA_BUTTERFLY_TCDM_SLAVE/butterfly_reg_top.sv
- rtl/DMA_BUTTERFLY_TCDM_SLAVE/butterfly_tcdm_top.sv
```

8. Alias to be put in bashrc to automate bender update , hw build and later on compilation

Open bashrc : gedit ~/.bashrc

Paste the alias (Adjust name and paths to match your environment):

```
build_tcdm_slave() {
   echo "========================================="
   echo " [1/3] Updating Bender Manifest & Scripts"
   echo "========================================="
   cd ~/GP || return

   # CRITICAL: Forces Bender to find new/modified files in Bender.yml
   # (Uncommented because we just modified Bender.yml to fix the file collision)
   ./bender update || return

   # Generate the fresh QuestaSim compile order
   ./bender script vsim > sim/compile.tcl || return

   echo " ▶️ Applying environment fixes..."
   ./fix_bugs.sh || return 1

   echo "========================================="
   echo " [2/3] Compiling Hardware (QuestaSim)"
   echo "========================================="
   cd sim || return

   # THE NUKE AND PAVE: Destroys old compiled modules so vopt sees your new interface
   echo " ▶️ Clearing old cached modules..."
```

```
    rm -rf work/
    make clean

    echo "▶️ Building hardware..."
    make build || return

    echo "▶️ Running vopt optimization..."
    vopt +acc=npr -o vopt_tb tb_pulp -floatparameters+tb_pulp -work work || return

    echo "========================================="
    echo " [3/3] Hardware Build Complete!"
    echo "========================================="

    # Uncomment these when you are ready to test the C driver
    #echo "▶️ Compiling and running TCDM C test..."
    #cd ~/GP/examples/TCDM_SLAVE_BUTTERFLY_TEST || return
    #make clean all
    #make run gui=1
}
```

Then source ~/.bashrc

And run build_tcdm_slave

SW integration steps same as in full ip integration(axi) files are on github and you already generated the header or re generate it (reggen). Notice that you have to use same address region (base , end).